

* Javascript :

* DOM → Document Object Model

→ console.log(window)

→ console.log(window.document)

OR

{ console.log(document) → gives all HTML & other web page related codes
console.dir(document) → gives hidden info =

→ document.getElementById(elem-Id)

↳ to access any element by its Id.

→ document.getElementById("title") → <h1 id="title"

→ const elem = document.getElementById("title")

→ elem.innerHTML → gives all the HTML tags and text

→ elem.innerText → returns text that is visible on the page, ignoring text hidden by CSS

→ elem.textContent → returns all the text within element including hidden text and text inside <script> and <style> tags

Note:

→ `document.getElementsByClassName(...)` → returns HTML Collection

→ document.querySelectorAdd(...) → return NodeList

→ Nodelist can implement forEach()

HTML Collection cannot

→ So convert them to an Array[] → how?

↳ using `Array.from()`

CSS Selectors : →

→ Element selector →

```
element_name {  
    . . .  
}
```

→ styling you want to add

→ Id Selector → # id {

→ class selector →

↓

specific elem
in that class

```
.className {  
    ---  
    ---  
}
```

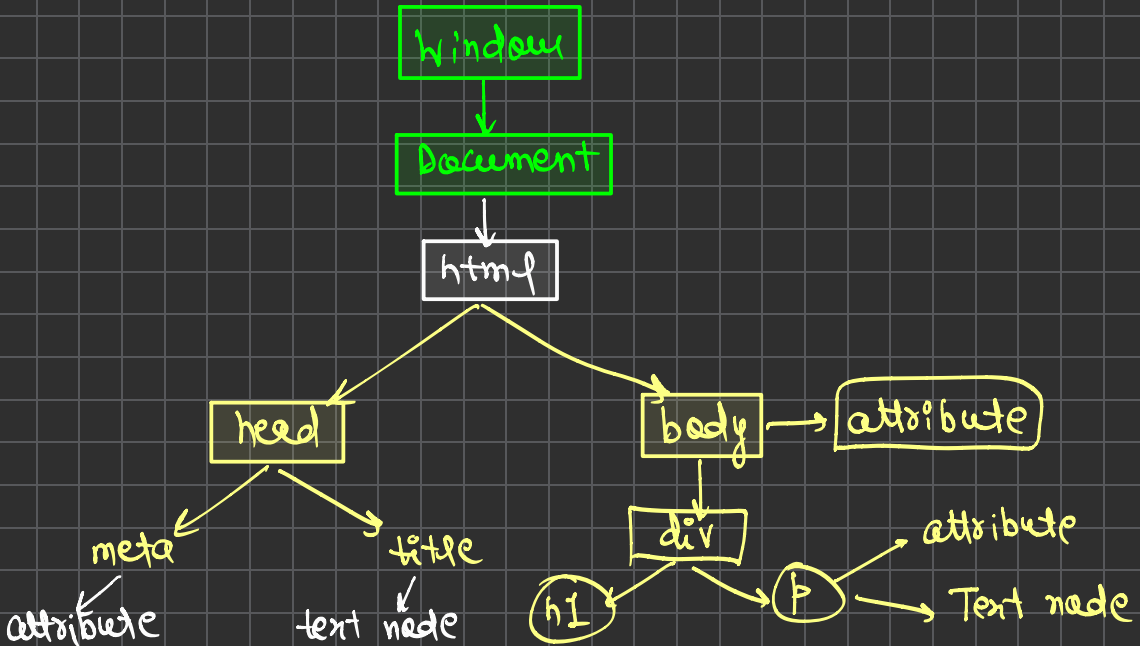
```

↓
p.className {
  ...
}

```

→ all `<p>` elements
are affected acc. to
class → className

➤ DOM tree structure: →



⊛ DOM objects
Creation & Accessing

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM | Chai aur Code</title>
</head>
<body style="background-color: #212122; color: #fff;">
  <div class="parent">
    <div class="days">Monday</div>
    <div class="days">Tuesday</div>
    <div class="days">Wednesday</div>
    <div class="days">Thursday</div>
  </div>
</body>
<script>
  const parent = document.querySelector('.parent')
  // console.log(parent.children);
  // console.log(parent.children[1]);
  // console.log(parent.children[2].innerText); → Wednesday

```

➤ `parent.children` → gives a HTMLCollection of all children nodes or elements.

➤ `parent.firstElementChild`

→ `const dayOne = parent.firstElementChild`

→ `dayOne.parentElement`

→ `dayOne.nextElementSibling`

➤ `parent.childNodes` → returns NodeList

↳ even a line break in the doc. is considered an individual node

↳ because entire doc. is repr. using a tree structure

↳ complex

Creating and appending an element:

➤ `const div = document.createElement('div')`
using `'.'` operator you can add properties to this element such as `id`, `class`, some styling,
→ also you can use `document.setAttribute('title', ...)` etc.

→ Now we have to attach/append this element to our document at a specific location in our doc.

⇒ Add some text to it before appending

→ `const textNode = document.createTextNode("Hello")`

→ `div.appendChild(textNode)`

→ `document.body.appendChild(div)`

Editing of element: →

→ `<ul class="language">`
 `<li>JS </li>`
`</ul>`

→ `function addLanguage(langName) {`
 `const li = document.createElement('li')`
 `li.innerText = ...`
 `document.querySelector('.language').appendChild(li)`
}

or you can directly append a Text Node to 'li' elem
then append it to the 'ul'.

for accessing child elements: →

→ `.querySelector("li:nth-child(3)")`
`("li: first-child")`
`("li: last-child")`

also there are `replaceWith()` → for replacing
`&` `remove()` → for deletion

* Events

→ `.onclick = function() {`
`alert(...)`
`}`

→ `.addEventListener('click', function() {` } `false` ^{by default}
_{or true}

→ `false` → Event Bubbling → from bottom to top

→ `true` → Event Capturing → top to bottom

→ `e.stopPropagation()` → to stop bubbling or capturing

→ `e.preventDefault()` → to prevent default behaviour

⊕ Async JS

→ by default in JS → Synchronous
→ Single threaded
 ↳ one call stack()
 ↳ one main thread

exact order
↑
one statement at a time

→ Execution Context

env. containing everything to run that code
↳ execute one line of code at a time

→ console.log → 1
→ console.log → 2

Call stack Memory heap

→ Executional context has 2 phases: { every fn. call() has its own EC }

→ Memory creation phase → mem is allocated for var, ...

→ execution phase → executing one line at a time.

Blocking code & Non blocking code

↓
block flow of program

↓
Read file sync

↓
doesn't block of execution

↓
Read file async